

MACHINE LEARNING PROJECT REPORT

BBC News Text Classification

Multinomial Logistic Regression with TF-IDF Feature Engineering

Prepared by: **Aleeza Nadeem**

Domain: Natural Language Processing / Text Classification

Date: August 2026

Executive Summary

This report documents the development of a multinomial text classification model that automatically sorts BBC News articles into one of five editorial categories: business, entertainment, politics, sport, and technology. Built on a TF-IDF representation of cleaned article text and a multinomial Logistic Regression classifier, the model reaches 96.18% accuracy on a held-out test set, with the training-versus-testing accuracy gap kept to a modest 3.56 percentage points.

96.18% TEST ACCURACY	99.74% TRAIN ACCURACY	5,000 TF-IDF FEATURES	5 NEWS CATEGORIES	1,440 TRAINING ARTICLES
--------------------------------	---------------------------------	---------------------------------	-----------------------------	-----------------------------------

1 The Problem This Model Solves

News organizations, aggregators, and content platforms publish and receive a continuous stream of articles that need to be organized before readers can find them. Manually tagging every incoming article with the correct section — business, entertainment, politics, sport, or technology — is slow, expensive, and inconsistent once the volume of content grows beyond a small editorial team.

This project addresses that bottleneck by building an automated text classifier that reads the raw text of a news article and predicts which of the five BBC News categories it belongs to. In a real newsroom or content-management pipeline, a model like this can:

- Automatically route incoming articles or wire-service feeds to the correct section, reducing manual tagging effort.
- Power content recommendation and search filters on a news website by reliably labelling unstructured text at scale.
- Flag mis-categorized or ambiguous articles for editorial review by comparing the model's predicted category against the section an article was published in.
- Act as a lightweight baseline for downstream tasks such as topic-based alerting, archival indexing, or trend analysis across news categories.

Why this matters

The core challenge is a classic supervised NLP problem: converting unstructured, variable-length text into a numerical representation a model can learn from, then training a classifier that generalizes to articles it has never seen — including the fully unseen Kaggle test set used later in this report.

2 Dataset Overview

The dataset originates from the BBC News classification competition and is provided as two files: a labelled training set (BBC News Train.csv) and an unlabelled test set (BBC News Test.csv) used purely for final inference. Each row contains an article ID, the raw article text, and — for the training file — a category label.

Attribute	Value
Initial training rows	1,490
Rows after removing duplicates	1,440
Missing values found	0 (Text and Category)
Categories	business, entertainment, politics, sport, tech
Unseen test articles (Kaggle)	735
Average article length	377 words
Shortest / longest article (post-cleaning)	89 / 3,266 words

Table 1 — Dataset composition before and after cleaning.

3 Data Cleaning & Preprocessing

3.1 Data Hygiene

The raw dataset was first checked for missing values and duplicate articles. No missing values were found in the Text or Category columns, but 50 exact duplicate articles were identified and removed, reducing the dataset from 1,490 to 1,440 rows. Removing duplicates before splitting the data is essential — otherwise the same article text could appear in both the training and test partitions, artificially inflating reported accuracy through data leakage.

3.2 Text Normalization

A custom cleaning function was applied to every article to produce a consistent, lowercase, punctuation-free version of the text:

- Lowercasing — so that "Government" and "government" are treated as the same token.
- Removing punctuation and digits via the regular expression `[\^a-z\s]` — isolating alphabetic content only.
- Collapsing repeated whitespace into single spaces and trimming leading/trailing space.

This normalization ensures the downstream TF-IDF vectorizer maps semantically identical words to the same feature, rather than fragmenting the vocabulary across capitalization and punctuation variants.

3.3 Outlier Detection

Word counts were computed for every cleaned article to check for degenerate or corrupted rows. Summary statistics showed a mean article length of 377 words (std. 209), ranging from 89 to 3,266 words. Articles shorter than 10 words — a threshold chosen to filter out empty or corrupted rows — were removed; no articles in this dataset fell below that threshold, so all 1,440 rows were retained. Unusually long articles were deliberately kept rather than truncated, since L2 normalization in the TF-IDF step (Section 5) automatically rescales longer documents rather than requiring manual trimming.

4 Exploratory Data Analysis

Before modelling, the class balance and article-length distribution were examined to understand whether the dataset would require any correction for imbalance. The test partition (a stratified 20% split, detailed in Section 6) contains the following distribution:

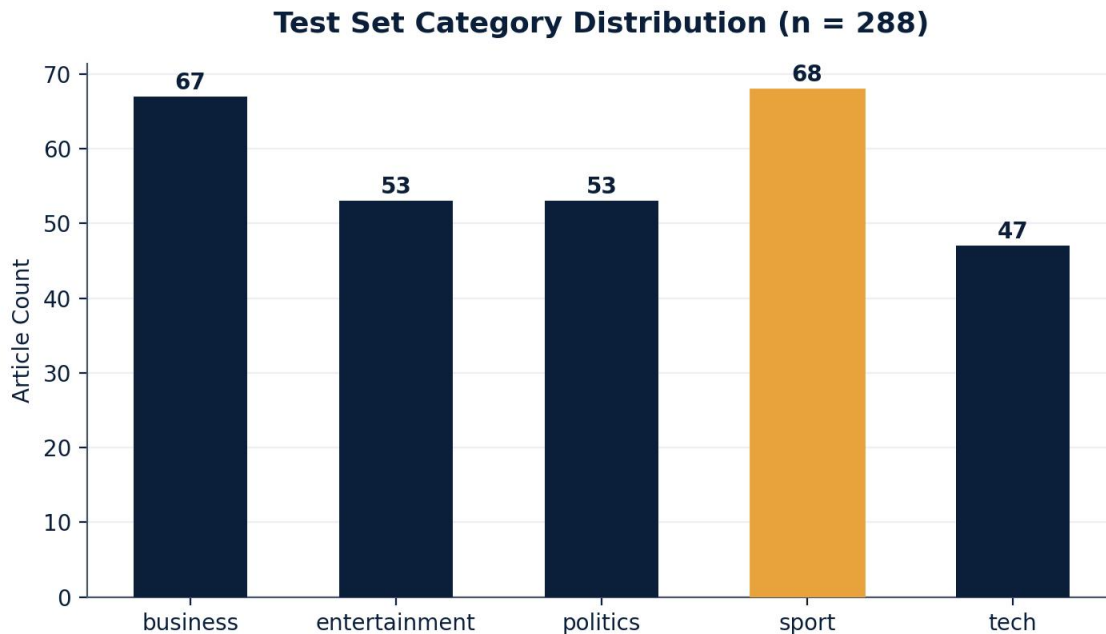


Figure 1 — Category distribution on the held-out test set (n = 288). Class sizes range from 47 (tech) to 68 (sport) articles.

The classes are reasonably balanced — no category dominates the dataset, and the ratio between the largest and smallest class is under 1.5:1. This is favourable for a baseline multinomial classifier, since severe class imbalance would otherwise require resampling or class-weighting to avoid the model defaulting to the majority class.

5 Feature Engineering: TF-IDF Vectorization

Raw text cannot be fed directly into a Logistic Regression model — it must first be converted into a numerical matrix. This project uses Term Frequency–Inverse Document Frequency (TF-IDF), which weights each word (or word pair) in an article by how often it appears in that article, offset by how common it is across the entire corpus. Common filler words score low; distinctive, topic-specific words score high.

Parameter	Setting	Purpose
stop_words	'english'	Removes filler words (the, and, of) that carry no topical signal
max_features	5,000	Caps vocabulary size to the 5,000 most informative terms
ngram_range	(1, 2)	Captures both single words and two-word phrases (e.g. "interest rate")

norm	'l2'	Rescales each document vector to unit length, neutralizing article-length differences
-------------	------	---

Table 2 — TF-IDF vectorizer configuration.

Critically, the vectorizer is fit only on the training partition and then used to transform both the training and test sets. Fitting on the full dataset before splitting would leak vocabulary and document-frequency statistics from the test set into training, producing an overly optimistic accuracy estimate. The resulting training matrix has shape $(1,152 \times 5,000)$ — 1,152 training articles represented across a 5,000-term vocabulary.

6 Train-Test Split Strategy

The cleaned dataset was split into training and testing partitions using an 80/20 stratified split (`random_state = 42`). Stratification ensures the proportion of each news category is preserved in both partitions, which is particularly important given the natural imbalance between categories such as tech (fewer articles) and sport (more articles).

Partition	Article Count	Share
Training set	1,152	80%
Testing set	288	20%

Table 3 — Train/test split sizes.

7 Model Architecture

The classifier is a multinomial (softmax) Logistic Regression model — a linear model that, unlike a one-vs-rest approach, jointly estimates probabilities across all five categories at once using the softmax function. It was trained with the L-BFGS solver, a quasi-Newton optimization method well suited to the high-dimensional but sparse TF-IDF feature space.

Hyperparameter	Value
Solver	lbfgs
Multi-class strategy	multinomial (softmax)
Max iterations	1,000
Random state	42
Input features	5,000-dimensional TF-IDF vectors
Output classes	5 (business, entertainment, politics, sport, tech)

Table 4 — Logistic Regression configuration.

Logistic Regression is a strong, interpretable baseline for text classification: with a sparse, high-dimensional TF-IDF input, a linear decision boundary is often sufficient to separate topically distinct classes, and the model trains quickly enough to iterate on preprocessing and vectorization choices.

8 Results & Evaluation

8.1 Overall Test Accuracy

On the 288-article held-out test set, the model achieved an overall accuracy of 96.18%. This means the model correctly predicted the news category for roughly 277 out of every 288 unseen articles.

8.2 Per-Class Performance

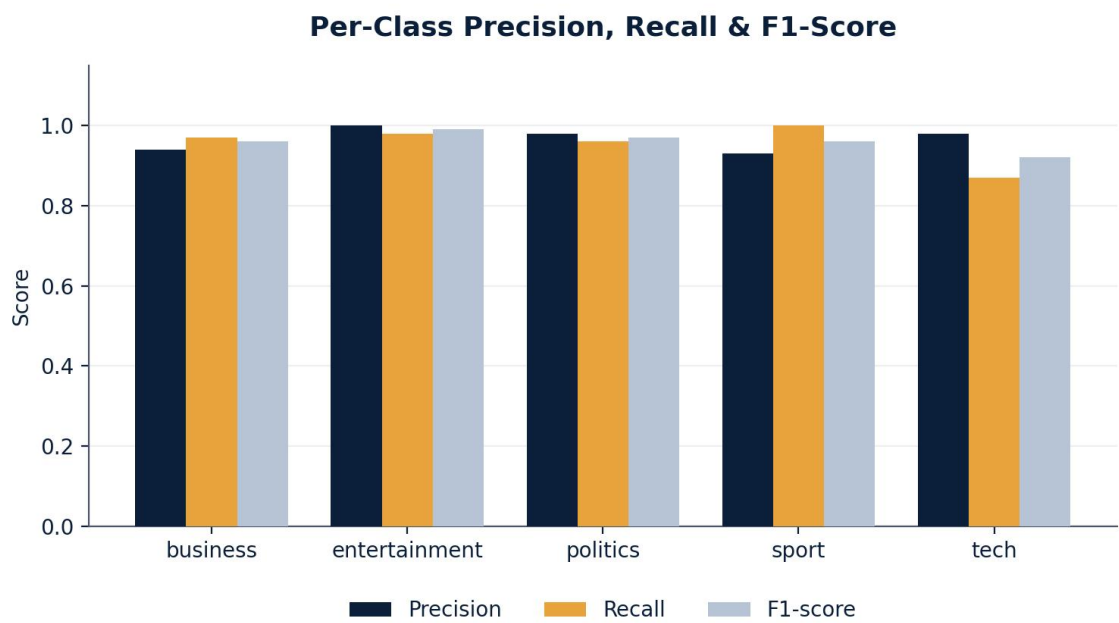


Figure 2 — Precision, recall, and F1-score by category.

Category	Precision	Recall	F1-Score	Support
business	0.94	0.97	0.96	67
entertainment	1.00	0.98	0.99	53
politics	0.98	0.96	0.97	53
sport	0.93	1.00	0.96	68
tech	0.98	0.87	0.92	47
macro avg	0.97	0.96	0.96	288
weighted avg	0.96	0.96	0.96	288

Table 5 — Full classification report on the test set.

Entertainment is classified almost perfectly ($F1 = 0.99$), and sport achieves perfect recall — every sport article in the test set was correctly identified, though at a slight cost to precision (0.93), meaning a small number of non-sport articles were mislabelled as sport. The weakest category is tech, with recall of only 0.87: roughly 13% of technology articles were misclassified into another category, most likely business or science-adjacent politics coverage where vocabulary overlaps.

9 Generalization Check: Train vs. Test Accuracy

To confirm the model generalizes rather than memorizing the training data, training and testing accuracy were compared directly.

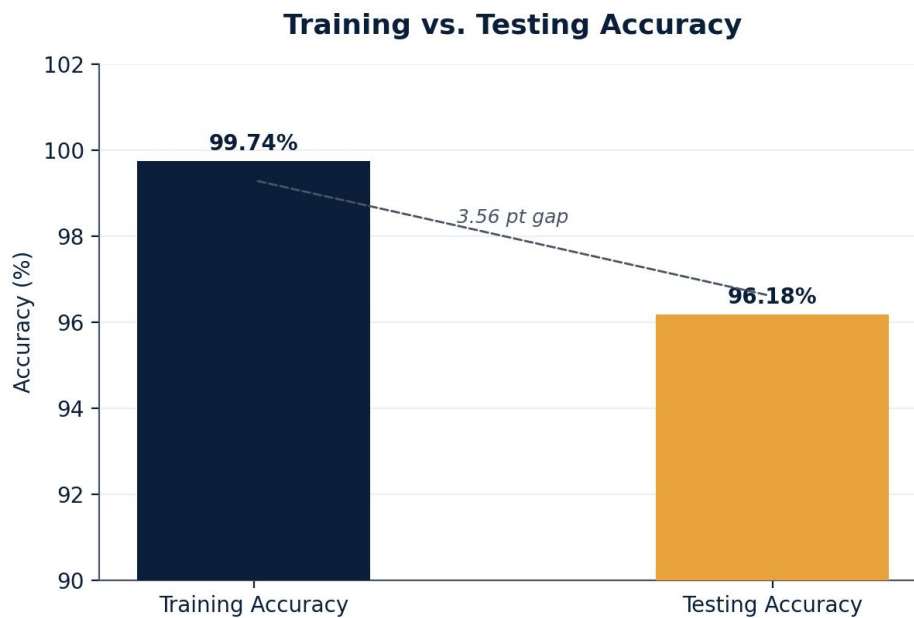


Figure 3 — Training accuracy (99.74%) vs. testing accuracy (96.18%).

A 3.56-percentage-point gap between training and testing accuracy indicates mild overfitting, which is expected and acceptable for a high-dimensional, sparse linear model of this kind — the 5,000-feature TF-IDF space gives the model enough capacity to fit training-specific vocabulary patterns almost exactly (99.74%), while still transferring well to unseen articles (96.18%). The gap is small enough that no additional regularization was deemed necessary for this baseline, though L2 regularization strength (the C parameter) would be the first lever to tune if the gap were larger.

10 Inference on Unseen Data (Kaggle Test Set)

Beyond the internal train/test split, the trained model was applied to a fully separate, unlabelled set of 735 articles (BBC News Test.csv) to simulate real-world deployment. The same cleaning function used on the training data was applied to this set, and the already-fitted TF-IDF vectorizer was used in transform-only mode — never re-fit — to avoid any data leakage from this new data back into the model.

ArticleId	Predicted Category
1018	sport
1319	tech
1138	sport
459	business
1020	sport
51	sport
2025	politics
1479	politics
27	entertainment
397	business

Table 6 — Sample of predictions generated for the unseen Kaggle test set (735 articles total, saved to submission.csv).

11 Discussion & Key Insights

- **Duplicate removal and missing-value checks before any modelling step prevented data leakage and wasted compute — a small but essential hygiene step often skipped in quick prototypes.**
- Lowercasing and regex-based punctuation stripping ensured identical words map to a single token, maximizing the effective density of the TF-IDF vocabulary.
- Fitting the TF-IDF vectorizer strictly on the training partition — and only transforming the test and unseen sets — is the single most important safeguard against inflated accuracy in text classification pipelines.
- L2 normalization inside TfidfVectorizer automatically down-weights long articles by rescaling every document vector to unit length, removing the need to manually cap or truncate the 3,266-word outlier identified during EDA.
- The unigram + bigram feature space (ngram_range=(1,2)) lets the model pick up two-word phrases such as "interest rate" or "prime minister" that carry more topical signal than either word alone.
- A stratified split kept category proportions consistent between training and testing, which is what makes the per-class precision/recall numbers in Table 5 directly comparable to the overall dataset composition.

12 Limitations & Future Work

- **Linear ceiling:** Logistic Regression on TF-IDF cannot capture word order or deep contextual meaning; embedding-based models (e.g. word2vec, or a fine-tuned transformer such as DistilBERT) would likely close some of the tech-category recall gap.
- **Vocabulary cap:** capping the vocabulary at 5,000 features may exclude rarer but discriminative terms; a higher max_features or sub-word tokenization could be tested.
- **No hyperparameter search:** the C (inverse regularization strength) parameter was left at its default; a grid or randomized search could tighten the 3.56-point train/test gap further.

- Category overlap: business, politics, and tech articles can share vocabulary (e.g. "regulation", "investment"), which likely explains the confusions behind tech's lower recall — a confusion-matrix-driven error analysis would help target which category pairs are most conflated.

13 Conclusion

This project delivers a clean, leakage-safe pipeline that takes raw BBC News article text through cleaning, outlier filtering, TF-IDF vectorization, and multinomial Logistic Regression classification, reaching 96.18% test accuracy across five news categories. The approach directly addresses the real-world need for automated article routing and tagging in a newsroom or content platform, and the small train/test variance gap (3.56 points) shows the model generalizes well rather than memorizing the training set. The resulting classifier — together with its predictions on the fully unseen 735-article test set — provides a solid, interpretable baseline that future iterations can extend with richer features or deep-learning architectures.